Chit Chat service - Mandatory Activity 3

By: Rasmus, Thor & Valdemar

23 oktober 2026

1 Streaming Choice

In our version of Chit Chat, we opted for using bidirectional streaming, as the nature of a chat service is to send messages in both directions, both to the server when a client wants to send a message, and from the server, when receiving other clients' messages. If we instead had chosen client-side streaming, the client would still be able to send messages to the server, but would not receive anything back at all. This would not satisfy the S4 specification, where every message received by the server is broadcast to every live client. We could have used a server-side streaming approach, but that would require at least two rpc methods. One to connect to the server with server-side streaming (called listen, for example), and one unary rpc method that sends a message from the client to the server (called sendMessage, for example). It would also make detecting disconnects more difficult, as the server would have to separately find the listen routine and shut it down, whenever a user has chosen to leave.

2 System Architecture

To complete the mandatory activity Chit Chat we opted for a server-client architecture, because we found it to be the best architecture for out project. The reason for this is that the architecture describes exactly the functionality of our program and has a very high correspondence to our project, since our program consists of a server that clients connect to in order to exchange messages.

We could've also opted for a peer-to-peer architecture, but by using the architecture we would not have a centralized server. This could have caused some problems, for example if all peers disconnect, then there would not be any nodes to keep track of the logical time, which is a key aspect of the program. By using a server-client architecture the logical time is always centralized and being kept track of.

3 RPC Methods

In order to meet the requirements of the mandatory activity we were required to use Remote Procedure Calls (RPC from here on). We implemented the RPC method `chat`, which receives

a stream of client messages from the client and returns a stream of server messages to the client. This was done by implementing the two message types `ClientMessage` and `ServerMessage`.

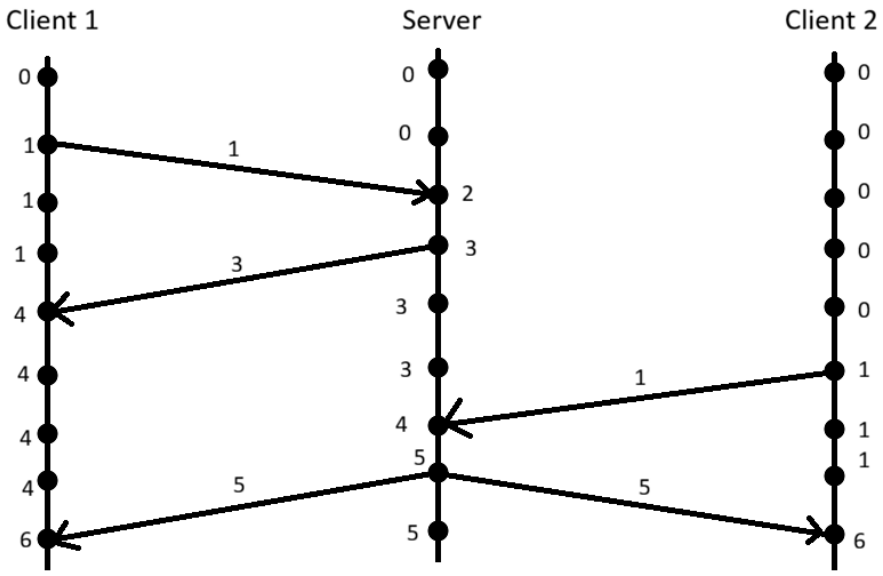
The two message types are very similar with the only difference being the `ClientMessage` having two boolean fields indicating whether a user is joining, leaving, or neither. Apart from that they have the same fields which are the clients ID, the lamport clock and the message content.

4 Timestamp Calculations

To implement lamport timestamps we gave our server and each client their own lamport, which would increment upon making a receiving/sending a message, or taking a local action.

When a message is sent, the sender sends their own current lamport timestamp along with it. This is then used when the server or any client eventually receives that message, as they then compare their lamport timestamp to the logical clock given in the message, picking the highest among the two as their new timestamp, incrementing it afterwards.

5 Diagram



Figur 1: Lamport clock example.

6 Appendix

Server and client logs can be found at:
https://github.com/rasmusmappel/RTVMandatoryActivity3/tree/main/logs